

Javascript: sintaxi, funcions, arrays, objectes

Anton Dalmau Mines

Índex

Introducció	5
Resultats d'aprenentatge	7
1 Sintaxi del llenguatge	9
1.1 Arquitectura de les aplicacions web. Javascript	9
1.1.1 Introducció	9
1.1.2 Naixement de JavaScript	13
1.2 Instal·lació de Node.js i npm	14
1.2.1 Instal·lació de Node.js	15
1.3 Sintaxi del llenguatge: variables, operadors, condicionals, bucles	16
1.3.1 Variables i tipus	16
1.3.2 Capturar dades	18
1.3.3 Operadors	19
1.3.4 Classes Core i Mòduls de JavaScript	22
1.3.5 Condicionals	26
1.3.6 Bucles (Iteracions)	27
2 Programació amb funcions	31
2.1 Funcions predefinides del llenguatge	31
2.2 Funcions definides per l'usuari	33
2.2.1 Declaració de funció	33
2.2.2 Funcions anònimes i assignació a variables	34
2.2.3 Funcions fletxa (Arrow Functions)	34
2.2.4 Paràmetres	36
2.2.5 Àmbit (Scope) de les variables	36
2.3 Conceptes avançats de funcions	37
2.3.1 Closures (funcions tancades)	37
2.3.2 Funcions com a objectes	38
2.3.3 Funcions com a fàbriques (closures + mètodes)	40
2.3.4 El context "this"	41
3 Programació amb arrays i objectes	43
3.1 Arrays. Objectes definits pel programador	43
3.1.1 Arrays com estructures de dades	43
3.1.2 Objectes com estructures de dades	45
3.1.3 Estructures de dades complexes	48
3.2 Programació orientada a objectes	50
3.2.1 Introducció a les classes	50
3.2.2 El paper de 'this' dins de les classes	51
3.2.3 Encapsulació i modificació de valors	51
3.2.4 Abstracció	53
3.2.5 Herència entre classes	54
3.2.6 Polimorfisme	55

3.2.7	Una menció als prototips	55
3.3	Exemples pràctics	56
3.3.1	Exemple 1: Estudiants i qualificacions	56
3.3.2	Exemple 2: Jerarquia d'animals	58

Introducció

Aquesta unitat introdueix els fonaments del llenguatge JavaScript, que constitueix la base de la programació al costat client en aplicacions web. S'hi treballa la sintaxi bàsica del llenguatge, la declaració de variables, els tipus de dades i les estructures de control, que permeten definir el flux d'execució d'un programa. Un element clau de la unitat és l'ús de funcions, tant com a mecanisme de reutilització de codi com a eina fonamental per estructurar programes més complexos. També s'introdueixen els *arrays*, que permeten gestionar col·leccions de dades, i els objectes, essencials per modelar informació de manera estructurada. Al llarg de la unitat es posa èmfasi en la comprensió del funcionament intern del llenguatge i en l'escriptura de codi clar i llegible. Aquest coneixement és imprescindible per afrontar la resta d'unitats del curs, ja que proporciona les eines bàsiques necessàries per treballar amb esdeveniments, el DOM i la programació asíncrona.

Resultats d'aprenentatge

En finalitzar aquesta unitat, l'alumne/a:

1. Selecciona les arquitectures i tecnologies de programació sobre clients web, identificant i analitzant les capacitats i les característiques de cadascuna.

- Caracteritza i diferencia els models d'execució de codi al servidor i al client web.
- Identifica les capacitats i els mecanismes d'execució de codi dels navegadors web.
- Identifica i caracteritza els principals llenguatges relacionats amb la programació de clients web.
- Identifica i caracteritza els principals llenguatges relacionats amb la programació de clients web.
- Reconeix les particularitats de la programació de guions i els seus avantatges i desavantatges sobre la programació tradicional.
- Verifica els mecanismes d'integració dels llenguatges de marques amb els llenguatges de programació de clients web.
- Reconeix i avalua les eines de programació i prova sobre clients web.

2. Escriu sentències simples, aplicant la sintaxi del llenguatge i verificant la seva execució sobre navegadors web.

- Selecciona un llenguatge de programació de clients web a funció de les possibilitats.
- Utilitza els diferents tipus de variables i operadors disponibles al llenguatge.
- Identifica els àmbits d'utilització de les variables.
- Reconeix i comprova les peculiaritats del llenguatge respecte a les conversions entre diferents tipus de dades.
- Utilitza mecanismes de decisió en la creació de blocs de sentències.
- Utilitza bucles i verifica el funcionament.
- Afegeix comentaris al codi.
- Utilitza eines i entorns per facilitar la programació, la prova i la documentació del codi.

3. Escriu codi, identificant i aplicant les funcionalitats aportades pels objectes predefinits del llenguatge.

- Identifica els objectes predefinits del llenguatge.
- Analitza els objectes referents a les finestres del navegador i els documents web que hi contenen.
- Escriu sentències que utilitzin els objectes predefinits del llenguatge per canviar l'aspecte del navegador i el document que hi conté.
- Genera textos i etiquetes com a resultat de l'execució de codi al navegador.
- Escriu sentències que utilitzin els objectes predefinits del llenguatge per interactuar amb l'usuari.
- Utilitza les característiques pròpies del llenguatge en documents compostos per diverses finestres.
- Utilitza mecanismes del navegador web per emmagatzemar informació i recuperar-ne el contingut.
- Depura i documenta el codi.

4. Programa codi per a clients web analitzant i utilitzant estructures definides per l'usuari.

- Classifica i utilitza les funcions predefinides del llenguatge.
- Crea i utilitza funcions definides per l'usuari.
- Reconeix les característiques del llenguatge relatives a la creació i l'ús de matrius (*arrays*).
- Crea i utilitza matrius (*arrays*).
- Utilitza operacions agregades per al maneig d'informació emmagatzemada a col·leccions.
- Reconeix les característiques d'orientació a objectes del llenguatge.
- Crea codi per definir l'estructura d'objectes.
- Crea mètodes i propietats.
- Crea codi que faci ús d'objectes definits per l'usuari.
- Utilitza patrons de disseny de programari.
- Depura i documenta el codi.

1. Sintaxi del llenguatge

En aquest capítol introduïrem la sintaxi bàsica del llenguatge **JavaScript**, el qual és un dels pilars fonamentals del desenvolupament web modern. Aprendrem com s'integra dins l'arquitectura de les aplicacions web, com instal·lar i configurar l'entorn de treball amb **Node.js** i **npm**, i com escriure les primeres instruccions i estructures bàsiques del llenguatge.

L'objectiu és entendre la forma i l'estructura del codi JavaScript, així com les convencions de nomenclatura i els elements essencials que utilitzarem en el curs.

1.1 Arquitectura de les aplicacions web. Javascript

En aquest primer bloc ens endinsarem en la sintaxi del llenguatge JavaScript, un dels pilars fonamentals del desenvolupament web modern. Abans, però, és important entendre en quin context apareix i per què és tan rellevant. La web actual ja no és un conjunt de pàgines estàtiques, sinó un ecosistema viu i dinàmic on les aplicacions interactuen constantment amb l'usuari i amb altres serveis. Per això, més enllà del simple maquetat amb **HTML** i l'estilització amb **CSS**, necessitem un llenguatge capaç d'aportar lògica, dinamisme i interactivitat: aquest llenguatge és **JavaScript**.

L'objectiu d'aquesta lliçó és proporcionar-nos una base sòlida sobre la sintaxi i el funcionament de JavaScript, començant per la seva ubicació dins l'arquitectura de les aplicacions web i seguint després amb els conceptes essencials que utilitzarem al llarg de tot el curs.

1.1.1 Introducció

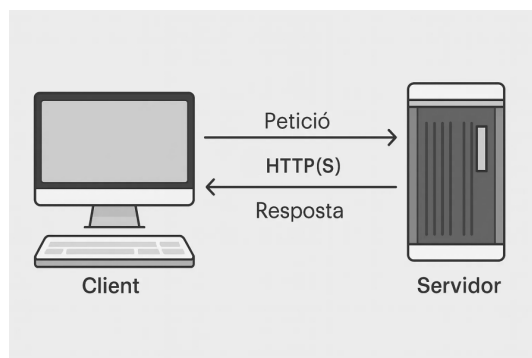
Avui en dia, tenim una gran quantitat de dispositius connectats al núvol. El portàtil, el telèfon mòbil, la tauleta, la televisió..., i amb l'*Internet of Things* (IoT), cada vegada aquests dispositius aniran en augment. Una de les coses habituals que fem amb algun d'aquests dispositius és navegar per la web tot visitant pàgines web.

D'altres dispositius i aplicacions, tot i que no serveixen per navegar per la web, sí que fan consultes a serveis web. Imaginem per exemple un aplicatiu web per controlar una casa domòtica, on podem posar la calefacció a distància. O una aplicació per al mòbil de calendari, que se sincronitza contínuament i ens avisa de quan tenim una reunió. Per tant, hem de prendre consciència del fet que el que ens permet la tecnologia web, avui dia, és molt més que visitar pàgines web.

Si una cosa tenen en comú tots aquests aplicatius web és que es basen en l'**arquitectura client-servidor**. Quan parlem de client web ens ve el cap el navegador web (Firefox, Chrome, Internet Explorer, etc.). Ara bé, una aplicació que ens permet consultar l'horari de la parada del bus també és un client web, tot i que la presentació no té format de pàgina web.

A l'altra banda tenim els servidors web, que serveixen el contingut web que tenen allotjat. Exemples de servidor web són l'Apache Web Server o l'Internet Information Server, però també el servidor d'aplicacions Tomcat (vegeu la figura 1.1).

FIGURA 1.1. Arquitectura client-servidor en la comunicació HTTP



Suposem que estem amb el nostre portàtil, i consultem la web de l'IOC des del navegador Chrome. Fixem-nos amb el camí que fan els paquets IP. Des del Chrome, en la barra de navegació, posem una URL que identifica unívocament el contingut que volem baixar (per exemple, www.ioc.xtec.cat). Aquesta informació es troseja en forma de paquets IP que viatgen per Internet. Els servidors DNS localitzen la direcció IP del servidor on està allotjat el contingut. A més, aquests paquets han de passar per *routers*, *firewalls* i diferents capes de seguretat.

Els paquets IP finalment s'ajunten en el destí, on tenim un servidor web Apache, per exemple, que allotja l'script *index.php*. Aquest script s'executa (es processa la part de PHP i s'ajunta amb la part d'HTML), i finalment el resultat se serveix al destinatari fent el camí invers.

Quan la pàgina web arriba al destí, en el navegador web podem visualitzar la informació demanada. El navegador web, el nostre client, és el responsable de maquetar correctament la pàgina web a partir del contingut HTML, els fulls d'estil CSS, i també el codi JavaScript que conté. Podem veure el codi font que hem rebut del servidor anant a l'inspector d'elements del navegador (fixem-nos que mai es veurà en el codi font el codi PHP; per contra, sí que podem veure el codi JavaScript).

L'entorn de client que estem estudiant és precisament la part del navegador web, en contraposició amb la part de servidor. Així com el codi PHP s'executa en el servidor, el codi JavaScript que contenen les webs s'executa en el client.

Als inicis d'Internet (anys 90 del s. XX), les pàgines web estaven formades únicament per documents **HTML estàtics**, que només permetien mostrar **text pla**

i **imatges**. No hi havia cap mena d'interactivitat amb l'usuari ni possibilitat de resposta dinàmica per part del navegador.

Amb el pas del temps, i especialment amb l'aparició de la **Web 2.0**, aquest panorama ha canviat radicalment. Les pàgines actuals són **dinàmiques, interactives** i amb una gran càrrega **visual**. Aquest canvi ha estat possible gràcies a la incorporació de **JavaScript**, un llenguatge de programació que s'executa directament en el navegador de l'usuari (cantó del client).

JavaScript ha esdevingut una **tecnologia clau** en el desenvolupament web modern per molts motius:

- **Llenguatge natiu dels navegadors.** JavaScript és **l'únic llenguatge** que tots els navegadors web entenen i poden executar de manera nativa. Per utilitzar altres llenguatges en el costat client (com TypeScript, Dart o Elm), és necessari utilitzar un **transpilador** que els converteixi prèviament a JavaScript.
- **Efectes visuals i dinamisme.** Permet aplicar una gran varietat d'**efectes visuals** que milloren l'experiència de l'usuari, com:
 - Aparició i desaparició de capes amb efectes de dissolució
 - Desplegables o continguts que es col·lapsen
 - Finestrals emergents (pop-ups)
 - Transicions d'imatges i galeries interactives
 - *Scrolls* animats, *sliders* i menús dinàmics
- **Interactivitat amb l'usuari.** JavaScript permet respondre a accions de l'usuari en temps real: clics, tecles, moviments del ratolí, etc. Això fa possible construir interfícies riques i intuïtives.
- **Validació de formularis en el client.** Amb JavaScript es pot validar la informació d'un formulari **abans d'enviar-la al servidor** (això millora la usabilitat, redueix el temps de resposta i evita càrregues innecessàries al servidor), per exemple:
 - Verificar el format d'un DNI, número de telèfon o correu electrònic
 - Comprovar que un camp no estigui buit
 - Validar dates o codis postals
- **Accés a informació del dispositiu i del navegador.** JavaScript pot accedir a dades del navegador, com:
 - Sistema operatiu
 - Idioma del navegador
 - Resolució de pantalla
 - Estat de connexió a Internet
- **Desenvolupament d'extensions i eines.** JavaScript és la base per crear **extensions** per navegadors com **Chrome**, **Firefox** o **Edge**, que permeten ampliar la funcionalitat del navegador amb eines personalitzades.

Transpilador

Un transpilador és una eina que converteix codi font escrit en un llenguatge de programació a un altre llenguatge de programació similar, mantenint habitualment el mateix nivell d'abstracció.

- **Aplicacions web avançades:** Gràcies a JavaScript i als seus *frameworks* (React, Angular, Vue,...), és possible crear **aplicacions web complexes** amb funcionalitats pròpies d'un programari d'escriptori. Alguns exemples destacats són:
 - **Google Docs** (processador de textos i fulls de càlcul en línia)
 - **Figma** (disseny d'interfícies col·laboratiu)
 - **Canva** (editor gràfic al núvol)
 - **Jocs:** per exemple, bit.ly/3hoeO7T

Tot i que **JavaScript** és un llenguatge de programació que pot executar-se fora del context web (com veurem més endavant amb Node.js), el seu gran èxit s'ha produït gràcies a la web. És una de les tecnologies fonamentals per desenvolupar **pàgines web dinàmiques i interactives**.

Per començar a treballar amb JavaScript dins d'una pàgina HTML, hem de veure com conviuen la **sintaxi HTML** i la **sintaxi JavaScript**. JavaScript s'integra en les pàgines web mitjançant l'etiqueta `<script>`.

L'atribut `type="text/javascript"` que sovint apareix en exemples antics ja no és necessari en HTML5, ja que JavaScript és el llenguatge de script per defecte.

Exemple bàsic de pàgina HTML i interacció amb l'usuari

El codi que veurem a continuació el podem escriure amb un editor de text o amb un IDE com per exemple Visual Studio Code. Després el podem desar dins d'una carpeta determinada.

```
1  <!DOCTYPE html>
2  <html lang="ca">
3    <head>
4      <meta charset="UTF-8">
5      <title>El meu primer exemple</title>
6      <script>
7        function actualitzaParagraf() {
8          document.getElementById("paragraf").innerText = "El contingut del
          paràgraf ha canviat!";
9        }
10     </script>
11   </head>
12   <body>
13     <h1>El meu primer exemple</h1>
14     <p id="paragraf">Aquest és el text original del paràgraf.</p>
15     <button onclick="actualitzaParagraf()">Clica'm</button>
16   </body>
17 </html>
```

Quan obrim aquesta pàgina en un navegador (per exemple Chrome o Firefox), veurem un paràgraf i un botó. Quan fem clic al botó, el contingut del paràgraf canviarà. Aquest és el comportament bàsic que permet JavaScript: **interacció amb l'usuari**.

Explicació del funcionament:

- El botó HTML incorpora l'atribut `onclick`, que **dispara un esdeveniment** quan l'usuari hi fa clic.
- Aquest esdeveniment executa la funció `actualitzaParagraf()`, definida dins de l'etiqueta `<script>`.
- La funció localitza l'element amb identificador `paragraf` (mitjançant `document.getElementById`) i modifica el seu contingut textual.

Ubicació de l'etiqueta `<script>`:

Tot i que en l'exemple anterior hem situat el codi JavaScript dins del `<head>`, també és **perfectament vàlid i sovint recomanable** col·locar-lo **al final del** `<body>`. Això assegura

que l'HTML estigui carregat abans que s'executi el codi JavaScript, evitant errors si l'script accedeix a elements que encara no existeixen en el DOM.

```
1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8">
5     <title>El meu primer exemple</title>
6   </head>
7   <body>
8     <h1>El meu primer exemple</h1>
9     <p id="paragraf">Aquest és el text original del paràgraf.</p>
10    <button onclick="actualitzaParagraf()">Clica'm</button>
11
12    <script>
13      function actualitzaParagraf() {
14        document.getElementById("paragraf").innerText = "El contingut del
15        paràgraf ha canviat!";
16      }
17    </script>
18  </body>
19 </html>
```

Dins l'etiqueta `<script>` s'hi troba el nostre codi. Sovint, sobretot si el codi es llarg, aquest s'allotja en un fitxer extern amb extensió `.js` i s'enllaça de la següent manera:

```
1 <script src="elMeuScript.js"></script>
```

Com provar el codi:

- 1) Obrirem un **editor de text pla** (com Notepad, Gedit, VS Code, Sublime Text3, etc.).
- 2) Copiem-hi l'exemple HTML anterior i el desarem com `exemple.html`.
- 3) Obrim aquest fitxer amb el navegador. Podem fer-ho arrossegant-lo al navegador, fent doble click sobre l'arxiu, amb el botó dret del ratolí i l'opció *obrir amb*, o escrivint la ruta al navegador:

```
1 file://ruta_al_fitxer/exemple.html
```

- 4) Veurem com, en clicar el botó, el contingut del paràgraf es modifica: la nostra **primera interacció JavaScript** ha tingut lloc!

Aquesta pràctica introdueix conceptes essencials com l'ús d'**esdeveniments**, **funcions**, i la **manipulació del DOM (Document Object Model)**, que són fonaments de qualsevol aplicació web moderna.

1.1.2 Naixement de JavaScript

JavaScript va ser creat per **Netscape** i, el 1997, es va proposar com a estàndard a l'**ECMA** (European Computer Manufacturers Association). Així va néixer **ECMAScript**, el llenguatge base de JavaScript, avui dia reconegut com a estàndard **ISO/IEC**.

Els problemes de compatibilitat entre navegadors (Internet Explorer, Netscape Navigator, Opera) van impulsar la necessitat d'estandardització. El **W3C (World Wide Web Consortium)** va establir el **DOM (Document Object Model)** per garantir un comportament coherent dels elements HTML/XML a tots els navegadors.

Avui en dia, **JavaScript és un dels llenguatges més importants i utilitzats del desenvolupament web**, i va molt més enllà de la seva funció original

com a eina per fer pàgines web interactives. Amb l'aparició de *frameworks* i biblioteques com **React**, **Angular**, **Vue** o **Svelte**, JavaScript ha esdevingut l'eix central per a la creació d'aplicacions web modernes, conegudes com a *Single Page Applications* (SPA). Aquestes aplicacions permeten experiències d'usuari fluides, amb actualitzacions dinàmiques del contingut sense necessitat de recarregar la pàgina.

A més, gràcies a **Node.js**, JavaScript també s'utilitza en el costat del servidor, fet que permet desenvolupar aplicacions completes amb un únic llenguatge, tant al frontend com al backend. Això ha propiciat l'aparició de noves arquitectures, com la **MERN stack** (MongoDB, Express, React, Node.js), molt popular en projectes web i *startups*. Amb eines com **npm** (*Node Package Manager*), la comunitat de desenvolupadors disposa d'un ecosistema immens de paquets i recursos que acceleren la creació d'aplicacions escalables, modernes i mantenibles.

1.2 Instal·lació de Node.js i npm

Tot i que **JavaScript** és conegut principalment pel seu ús en desenvolupament web, cal remarcar que **no és exclusiu de la web**. Es tracta d'un llenguatge de programació generalista, amb entorns d'execució independents del navegador que permeten desenvolupar tota mena d'aplicacions.

Quan s'executa JavaScript dins d'un navegador (com Firefox, Chrome o Safari), el codi és processat per un **motor de JavaScript** integrat en el mateix navegador (vegeu la taula 3.1).

TAULA 1.1. Motors de JavaScript destacats

Motor	Navegador / Entorn	Llenguatge d'implementació
SpiderMonkey	Firefox	C/C++
V8	Chrome, Edge, Node.js	C++
JavaScriptCore	Safari	C++
Chakra (antic)	Internet Explorer	C++

Una de les tecnologies més revolucionàries que han consolidat JavaScript fora del navegador és **Node.js**

Node.js és un entorn d'execució que utilitza el motor **V8** de Google (el mateix que fa servir Chrome) per executar codi JavaScript en el servidor. Va ser creat per Ryan Dahl el 2009, i permet utilitzar JavaScript per crear:

- Servidors web
- Aplicacions de xarxa (com xats en temps real)
- Scripts de línia d'ordres
- Automatitzacions i eines de desenvolupament

- APIs RESTful i serveis backend

Les característiques clau de Node.js són:

- Basat en **esdeveniments** i **programació asíncrona**
- Lleuger, ràpid i escalable
- Disposa del **gestor de paquets npm**, amb milers de biblioteques

1.2.1 Instal·lació de Node.js

A continuació, es detalla com instal·lar **Node.js** (i el seu gestor de paquets **npm**) sota la plataforma Windows (de manera anàloga es fa amb altres sistemes operatius):

1. Anem a la web oficial, nodejs.org.
2. Triem la **versió LTS (Long Term Support)** per garantir estabilitat.
3. Descarreguem i executem l'instal·lador.
4. Durant la instal·lació:
 - Acceptem els termes d'ús.
 - Marquem l'opció que afegeix Node.js al `PATH` (sol estar marcada per defecte).
 - Continuem endavant fins que finalitzi.
5. Un cop acabat, obrim una nova terminal i comprovem les versions que hem instal·lat:

```
1 node -v
2 npm -v
```

Així verifiquem que tant node com npm (el gestor de paquets) funcionen correctament.

Exemple bàsic amb Node.js

A continuació, es mostra un exemple simple d'un **servidor HTTP** escrit amb Node.js:

```
1 const http = require('http');
2
3 const server = http.createServer((req, res) => {
4   res.statusCode = 200;
5   res.setHeader('Content-Type', 'text/plain; charset=utf-8');
6   res.end('Hola món des de Node.js!');
7 });
8
9 server.listen(3000, () => {
10   console.log('Servidor en funcionament a <http://localhost:3000>');
11 });
```

Per executar aquest servidor:

- 1) Guardem el codi en un fitxer anomenat **server.js**.
- 2) Obrim una terminal en el directori on hem desat l'script `server.js` i executem: `node server.js`.

Visitem <http://localhost:3000> al navegador per veure la resposta.

1.3 Sintaxi del llenguatge: variables, operadors, condicionals, bucles

Un cop entès el paper de JavaScript dins de l'arquitectura web i coneguts els seus orígens, és el moment de centrar-nos en la seva **sintaxi bàsica**. La sintaxi és el conjunt de regles que defineixen com s'ha d'escriure i estructurar el codi perquè el navegador (o qualsevol entorn d'execució) el pugui interpretar correctament. Aprendre aquests fonaments és imprescindible per començar a programar, ja que constitueixen les peces bàsiques amb què construirem instruccions més complexes i aplicacions completes.

1.3.1 Variables i tipus

JavaScript és un llenguatge **feblement tipat**, és a dir, no indiquem de quin tipus és cada variable quan la declarem. Totes les variables poden contenir qualsevol tipus de dada i poden ser reassignades. Això té els seus avantatges i inconvenients.

Les variables són espais de memòria on emmagatzemem dades de manera temporal, i hi podem accedir en qualsevol moment durant l'execució del nostre programa. Hi ha diversos tipus i classes, que veurem tot seguit.

Per definir una variable en JavaScript, utilitzem la paraula reservada *let* i li donem un nom:

```
1 let elMeuValor;
```

També podem assignar-li un valor directament:

```
1 let dada = 5;
```

Com mostrar dades a l'usuari?

Per mostrar dades a l'usuari, tenim diferents mecanismes. Els més bàsics són mostrar-los per consola (`console.log(dada)`) o si ens trobem en un document web els podem mostrar per pantalla (`document.write(dada)`) o fins i tot fer-los sortir en una finestra emergent del navegador (`alert(dada)`).

O fer-ho en una línia posterior:

```
1 let dada;  
2 dada = 5;
```

Al llarg de les lliçons, sovint veurem declarar variables amb la partícula *const*. Això serveix per declarar una variable amb un valor constant i no podrà ser canviat excepte que la variable sigui de tipus objecte o array. Per exemple:


```
1 const PI = 3.1416;
```

Finalment, també val la pena conèixer que antigament es declaraven les variables amb la partícula `var`. Malgrat que encara funciona es desaconsella el seu ús.

Els noms de les variables haurien de ser el més **descriptius** possible per entendre millor què contenen. Han de començar sempre per una lletra, `$` o `_`, però **mai per números** ni altres caràcters especials. JavaScript distingeix entre majúscules i minúscules, per tant, `meuValor` i `MeuValor` serien dues variables diferents.

JavaScript disposa de **set tipus primitius** per emmagatzemar dades:

- **number**: s'utilitza per a **valors numèrics**, tant enters com decimals. Exemples:

```
1 let numEnter = 5;  
2 let numDecimal = 3.14;  
3 console.log(typeof numDecimal); // "number"
```

- **boolean**: només pot contenir `true` o `false`, i s'utilitza per representar estats:

```
1 let on = true;  
2 let off = false;
```

- **string**: representa cadenes de caràcters, entre cometes simples o dobles:

```
1 let missatge = "Hola món";  
2 let altreMissatge = 'Bon dia';
```

- **undefined**: És el valor que pren una variable quan **no se li ha assignat cap valor**, o s'ha assignat explícitament `undefined`:

```
1 let noDefinida;  
2 let tambéIndefinida = undefined;
```

- **null**: valor assignat que representa **absència intencionada de valor**:

```
1 let resposta = null;  
2 console.log(resposta); // null
```

- **bigint**: per treballar amb enters molt grans:

```
1 let granNumero = 1234567890123456789012345678901234567890n;  
2 console.log(granNumero); // →  
   1234567890123456789012345678901234567890n  
3 console.log(typeof granNumero); // "bigint"
```

- **symbol**: identificadors únics, sovint per propietats d'objectes:

```
1 const id = Symbol("usuariID");  
2 console.log(id); // Symbol(usuariID)  
3 console.log(typeof id); // "symbol"
```

Altres tipus

A part dels tipus primitius, JavaScript també utilitza **objectes** per representar estructures de dades més complexes. Tot en JavaScript és un objecte (incloses les funcions). Alguns objectes comuns són:

- `Object`
- `Array`
- `Date`

Tots ells hereten de la classe `Object`.

1.3.2 Capturar dades

Els valors desats en les variables estudiades a la secció anterior poden haver estat introduïts pels desenvolupadors o bé provenir directament de l'usuari. Capturar informació de l'usuari és fonamental per fer programes interactius i adaptatius. Tot i que la forma més habitual en aplicacions web són els formularis (que tractarem més endavant), hi ha altres mètodes senzills i molt útils per obtenir dades, tant en entorns de navegador com en NodeJS.

Des del navegador

En JavaScript que s'executa al navegador podem interactuar amb l'usuari de manera immediata a través de finestres emergents:

- **`prompt('Pregunta')`**. Permet mostrar un quadre de diàleg amb una pregunta i un camp de text per introduir dades. Retorna sempre una **cadena** de text, encara que l'usuari escrigui un número.

```
1 let nom = prompt('Com et dius?');  
2 console.log('Hola, ' + nom + '!');
```

- **`confirm('Estàs segur?')`**. Mostra un quadre de diàleg amb els botons **OK** i **Cancel**, retornant un valor **boolean** (`true` si l'usuari prem OK, `false` si prem Cancel).

```
1 let confirmar = confirm('Vols continuar amb l'acció?');  
2 if (confirmar) {  
3     console.log('Acció confirmada.');4 } else {  
5     console.log('Acció cancel·lada.');6 }
```

- **`alert('Missatge')`**. Tot i que no captura dades, és útil per mostrar informació immediata a l'usuari. Sovint es combina amb `prompt` o `confirm`.

```
1 alert('Benvingut a la nostra pàgina!');
```

Des de la consola amb NodeJS

En entorns fora del navegador, com NodeJS, podem capturar dades de l'usuari a través de la consola. La manera més senzilla és mitjançant el mòdul `readline`:

```
1 const readline = require('readline');
2
3 const rl = readline.createInterface({
4   input: process.stdin,
5   output: process.stdout
6 });
7
8 rl.question('Com et dius? ', (nom) => {
9   console.log('Hola, ${nom}!');
10  rl.close();
11 });
```

- `readline.createInterface` crea un canal per llegir i escriure dades a la consola.
- `rl.question` mostra la pregunta i espera que l'usuari introdueixi una resposta.
- La resposta es passa a una funció de *callback*, on podem processar-la i emmagatzemar-la en una variable.

A més, podem capturar dades de forma més avançada, com per exemple llegir múltiples preguntes en seqüència o processar entrades contínues, però això ja forma part d'usos més avançats que estudiarem més endavant.

1.3.3 Operadors

JavaScript disposa d'**operadors** per a treballar amb tipus primitius i objectes. Ens permeten formar expressions. Els més habituals són els **aritmètics**, però també n'hi ha d'altres.

Operadors aritmètics

Serveixen per fer càlculs matemàtics:

```
1 5 + 2    // Suma → 7
2 5 - 2    // Resta → 3
3 (3 + 2) - 5 // Parèntesis → 0
4 3 / 3    // Divisió → 1
5 6 * 3    // Multiplicació → 18
```

L'operador + també serveix per **concatenar** cadenes de text:

```
1 "Hola " + "món" + "!" // Resultat: "Hola món!"
```

També tenim operadors d'**increment** i **decrement** (pre i post):

```
1 let x = 1;  
2 let y = ++x; // x = 2, y = 2  
3 let z = y++ + x; // z = 4, y = 3, x = 1
```

Operador typeof

Aquest operador especial ens permet saber **quin tipus de dada** conté una variable. És un operador *unari* (s'aplica a un sol operand):

```
1 typeof 5 // "number"  
2 typeof false // "boolean"  
3 typeof "Joan" // "string"  
4 typeof undefined // "undefined"
```

Operadors booleans

Només tenen dos valors possibles: `true` i `false`. Però poden modificar-se amb diversos operadors:

Negació

Canvia el valor a l'oposat:

```
1 !true // false  
2 !false // true  
3 !!true // true (doble negació)
```

Igualtat i identitat:

- `==` → Igualtat feble (comprova si són iguals, permet conversió de tipus)
- `===` → Identitat estricta (comprova valor i tipus)

```
1 true === true // true  
2 true === false // false  
3 true !== false // true  
4 true !== true // false
```

Es recomana fer servir `===` i `!==` en comptes de `==` i `!=` per evitar errors inesperats.

Error comú

Un error comú és pensar que aquesta expressió és falsa:

```
1 25 == "25"
```

En realitat són iguals, però no idèntics. L'expressió que retornaria `false` seria:

```
1 25 === "25"
```

Comparació

Podem comparar valors amb:

```
1 5 > 3      // true
2 5 < 3      // false
3 3 >= 3     // true
4 2 <= 1     // false
5 "a" < "b" // true
```

Valors que es consideren 'false' en JavaScript:

Valors falsy en JavaScript

En JavaScript, quan fem servir una expressió dins d'una condició (if, while, etc.), qualsevol valor es converteix implícitament a `true` o `false`. Els valors que s'avaluen com a `false` s'anomenen **falsy values**.

Els únics valors que es consideren **falsy** són:

- `false` (el valor booleà fals)
- `0` i `-0` (el zero numèric)
- `""` (cadena buida)
- `null`
- `undefined`
- `NaN` (Not a Number)
- `0n` (BigInt zero)

Tots els altres valors es consideren **truthy** (encara que siguin sorprenents, com per exemple `"0"`, `[]` o `{}.`

Operadors lògics

AND

Només retorna `true` si **tots dos valors** ho són:

```
1 true && true   // true
2 true && false  // false
```

Aquest operador (`&&`) s'utilitza molt per verificar condicions sense modificar els valors. Per exemple, és útil per comprovar si una propietat existeix.

La lògica que segueix és la següent:

- Si el **primer valor** és `false`, es retorna aquest valor.
- Si el **primer valor** és `true`, es retorna el segon valor.

```
1 0 && true // Retorna: 0 → perquè el 0 es considera un valor "fals"
2 1 && "Hola" // Retorna: "Hola" → perquè l'1 (o qualsevol valor diferent de 0) es considera "cert"
```

OR

Retorna el **primer valor cert**, o false si tots són falsos:

```
1 true || false // true
2 false || false // false
3 process.env.PORT || 5000 // Si process.env.PORT no està definit, s'utilitza 5000 com a valor per defecte
```

1.3.4 Classes Core i Mòduls de JavaScript

A més dels **tipus primitius** com number, string, boolean i undefined, JavaScript disposa de **classes bàsiques incorporades** que ens permeten treballar amb estructures de dades més complexes. Aquestes classes formen part del nucli del llenguatge (Core).

Les més habituals són:

- Object
- Number
- Date
- Math
- String
- Array

Totes elles **hereten de Object** i tenen els seus propis **mètodes i propietats**.

Object

És la classe més genèrica. Un objecte és un conjunt de **propietats (clau-valor)**, i pot contenir també **funcions** (anomenades mètodes).

```
1 let llibre = {
2   titol: "JavaScript per a tothom",
3   autor: "Ada Codaire",
4   pàgines: 240,
5   llegit: false
6 };
7
8 llibre.titol // "JavaScript per a tothom"
```

```
9  llibre["pàgines"]    // 240
10 llibre.llegit = true; // assignació
```

Number

L'objecte Number representa **valors numèrics**, tant enters com decimals, i ofereix mètodes útils per manipular-los.

Creació de números:

```
1  let enter = 42;
2  let decimal = 3.1415;
3  let hexadecimal = 0xFF;    // 255 en decimal
4  let exponencial = 5.4e2;    // 540
```

També es pot crear amb el constructor:

```
1  let n = new Number(7);    // No gaire recomanable, millor forma
    primitiva
```

Valors especials:

```
1  1 / 0          // Infinity
2  -1 / 0         // -Infinity
3  "a" / 2        // NaN (Not a Number)
```

Mètodes de l'objecte Number:

```
1  let n = 2.5674;
```

- `n.toFixed(2)` → "2.57" (arrodoneix amb 2 decimals)
- `n.toExponential(2)` → "2.57e+0" (forma científica)
- `(15).toString(2)` → "1111" (binari)
- `(255).toString(16)` → "ff" (hexadecimal)

Els valors numèrics en JavaScript tenen un rang aproximat de:

- `Number.MAX_VALUE` → valor màxim ($\sim 1.79 \times 10^{308}$)
- `Number.MIN_VALUE` → valor mínim ($\sim 5 \times 10^{-324}$)
- `Number.NaN`, `Number.POSITIVE_INFINITY`, `Number.NEGATIVE_INFINITY`

Date

L'objecte Date ens permet **treballar amb dates i hores**.

Crear una data:

```
1 let ara = new Date();           // Data i hora actual
2 let dataEspecifica = new Date(2025, 6, 17); // 17 de juliol de
   2025 (mesos compten des de 0!)
```

Obtenir components d'una data:

```
1 ara.getFullYear(); // 2025
2 ara.getMonth();    // 6 (juliol)
3 ara.getDate();      // 17
4 ara.getDay();       // 4 (dijous, sent 0 = diumenge)
5 ara.getHours();     // hora actual
6 ara.getMinutes();   // minuts actuals
```

Format com a cadena:

```
1 ara.toString();      // "Thu Jul 17 2025 10:32:00 GMT+0200"
2 ara.toLocaleDateString(); // "17/7/2025"
```

Math

És un **objecte estàtic** que proporciona funcions i constants matemàtiques. No cal crear una instància (`new Math()` no s'utilitza).

Constants útils:

```
1 Math.PI      // 3.141592...
2 Math.E       // 2.718281...
```

Funcions matemàtiques bàsiques:

```
1 Math.round(6.7); // 7 (arrodoniment)
2 Math.floor(6.7); // 6 (arrodoniment a la baixa)
3 Math.ceil(6.1);  // 7 (arrodoniment a l'alça)
4 Math.abs(-5);    // 5 (valor absolut)
5 Math.pow(2, 3);  // 8 (potència)
6 Math.sqrt(16);   // 4 (arrel quadrada)
```

Aleatorietat:

```
1 Math.random(); // nombre aleatori entre 0 i 1
2 Math.floor(Math.random() * 10); // enter aleatori entre 0 i 9
```

Trigonometria i logaritmes:

```
1 Math.sin(Math.PI / 2); // 1
2 Math.cos(0);           // 1
3 Math.tan(Math.PI / 4); // ~1
4 Math.log(10);          // logaritme natural
```

Array

Un Array (matriu) és una **col·lecció ordenada de valors**. Aquests valors poden ser nombres, cadenes, booleans, objectes, altres arrays, etc.


```
1 let buida = [];  
2 let numeros = [1, 2, 3];  
3 let textos = ["Hola", "adeu"];  
4 let mesclat = [1, "dos", true, [4, 5]];
```

També es pot crear amb el constructor:

```
1 let altra = new Array(1, 2, 3);
```

En lliçons posteriors es treballaran els arrays de forma més exhaustiva.

String

Una String és una seqüència de caràcters. Es defineix amb cometes simples, dobles o *backticks* (template strings).

Creació de strings:

```
1 let salutacio = "Hola món";  
2 let nom = 'Joan';  
3 let complet = `Hola, ${nom}!`; // "Hola, Joan!"
```

Accés als caràcters:

```
1 salutacio[0]; // "H"  
2 salutacio.charAt(1); // "o"  
3 salutacio.length; // 8
```

Mètodes útils:

```
1 // Transforma el text a majúscules  
2 "Hola".toUpperCase(); // "HOLA"  
3 // Transforma el text a minúscules  
4 "Hola".toLowerCase(); // "hola"  
5 // Cerca un text donat dins una cadena de text i retorna 'true'  
  si el troba  
6 "Hola món".includes("món"); // true  
7 // Indica la primera posició de l'string donat dins una cadena  
  de text  
8 "Hola món".indexOf("m"); // 5  
9 // Retorna el text comprès entre el rang donat  
10 "Hola món".substring(0, 4); // "Hola"  
11 // Retorna el text sense els espais en blanc  
12 " text ".trim(); // "text"
```

Convertir un string en array:

```
1 let paraules = "un dos tres".split(" "); // ["un", "dos", "tres"]
```

1.3.5 Condicionals

Les estructures condicionals ens permeten **executar diferents blocs de codi** en funció d'una condició determinada. D'aquesta manera podem **controlar el flux del programa** segons diferents casos o estats.

Assignació condicional (Operador ternari)

És una forma **compacta** de fer una assignació en funció d'una condició:

```
1 condició ? valor_si_cert : valor_si_fals
```

Exemples:

```
1 let numero = 10;  
2 let mesGranQueCinc = numero > 5 ? "sí": "no";  
3 console.log(mesGranQueCinc) // sí
```

Sentència if

S'utilitza per executar codi només si una condició és certa:

```
1 let nota = 8;  
2  
3 if (nota >= 5) {  
4   console.log("Aprovat");  
5 }
```

Sentència if / else

Permet executar un bloc si la condició és **certa**, i un altre bloc si és **falsa**:

```
1 let nota = 8;  
2  
3 if (nota >= 5) {  
4   console.log("Aprovat");  
5 } else {  
6   console.log("Suspès");  
7 }
```

Sentència if / else if / else

Permet encadenar **diverses condicions**:

```
1 let nota = 8;  
2  
3 if (nota < 5) {  
4   console.log("Suspès");
```

```
5 } else if (nota < 7) {  
6   console.log("Bé");  
7 } else if (nota < 9) {  
8   console.log("Notable");  
9 } else {  
10  console.log("Excel·lent");  
11 }
```

Es van comprovant les condicions en ordre fins que se'n compleixi una.

Sentència switch

És una alternativa **més llegible** a múltiples if / else if. Comprova un valor i executa el bloc corresponent al cas:

```
1 let color = "groc";  
2  
3 switch (color) {  
4   case "vermell":  
5     console.log("vermell");  
6     break;  
7   case "groc":  
8     console.log("groc");  
9     break;  
10  case "verd":  
11    console.log("verd");  
12    break;  
13  default:  
14    console.log("El color no és ni vermell, ni groc, ni verd");  
15 }
```

Cal utilitzar *break* per evitar que s'executin els casos següents.

1.3.6 Bucles (Iteracions)

Els bucles ens permeten **repetir un bloc de codi** mentre es compleixi una condició. Són molt útils per **recórrer arrays**, repetir càlculs o generar estructures dinàmiques.

Tots els bucles tenen habitualment tres parts:

1. **Inicialització**: defineix la variable de control.
2. **Condició de permanència**: s'avalua abans o després de cada iteració.
3. **Actualització**: modifica la variable de control per avançar en el procés.

while

El bucle while executa el codi mentre la condició sigui **certa**:

```
1 let i = 1;
2
3 while (i <= 5) {
4   console.log(i);
5   i++;
6 }
7 // Resultat: 1 2 3 4 5
```

do...while

Aquest bucle **garanteix que el bloc s'executa com a mínim una vegada**, ja que la condició es comprova **després**:

```
1 let i = 1;
2
3 do {
4   console.log(i);
5   i++;
6 } while (i <= 5);
7 // Resultat: 1 2 3 4 5
```

for

El bucle for és el més habitual. Reuneix les tres parts en una sola línia:

```
1 for (let i = 0; i < 5; i++) {
2   console.log(i);
3 }
4 // Resultat: 0 1 2 3 4
```

Bona pràctica: evitar càlculs innecessaris

Evita repetir `.length` en cada iteració:

```
1 let array = [1, 2, 3, 4];
2
3 // Forma poc òptima
4 for (let i = 0; i < array.length; i++) {
5   console.log(array[i]);
6 }
7
8 // Forma òptima
9 for (let i = 0, len = array.length; i < len; i++) {
10  console.log(array[i]);
11 }
```

forEach (només per arrays)

És un **mètode dels arrays** que executa una funció per a cada element:

```
1 let valors = [10, 20, 30];
2
3 valors.forEach(function(valor, index) {
4   console.log("A la posició " + index + " hi ha: " + valor);
5 });
```

for...in (per a objectes)

Aquest bucle recorre **les propietats d'un objecte**:

```
1 let cotxe = {  
2   marca: "Toyota",  
3   model: "Yaris",  
4   any: 2020  
5 };  
6  
7 for (var clau in cotxe) {  
8   console.log(clau + ": " + cotxe[clau]);  
9 }  
10 // marca: Toyota, model: Yaris, any: 2020
```

També existeix for...of, que recorre valors directament (millor per a arrays, cadenes i col·leccions iterables).

```
1 for (let valor of [10, 20, 30]) {  
2   console.log(valor); // 10 20 30  
3 }
```